STANDARD OPERATING PROCEDURE

Discretionary FR Classification User Guide

How to run the tool, where files go, and what to do when something breaks

Audience: foundation staff running the tool day-to-day, and the technical maintainer who occasionally rebuilds it

Applies to: ECF Classification.exe and the funding-request classification pipeline

Organization: Edmonton Community Foundation

Last updated: August 28, 2026

> **What this tool does:** reads the free-text fields on discretionary funding-request workbooks and classifies each row by ethnic/cultural origin, gender identity, and sexual identity against ECF's taxonomy – then hands you the flagged, uncertain rows to review by hand before anything is treated as final.

1. What This Tool Does

You don't need to understand the code to use the tool, but it helps to know the shape of it. This section describes it in plain language – for the technical detail, see `README.md` sections 2–4.

> **Every decision follows a fixed rule, not a guess.** There is no AI or machine-learning model deciding your classifications. The same funding request text will always produce the same result, and every result can be traced back to the specific rule that produced it. Some machine-learning components exist in the codebase, but they are switched off in the path that actually writes your data – more on that in Section 10, for the maintainer.

The engine works in three stages, and it's worth knowing them because they explain why some rows get flagged and others don't:

Stage 1 — Scan for anything that might be relevant

The tool reads the project description, summary, purpose, and funding request name, and flags every word or phrase that could possibly indicate an identity group – a country name, a community term, an organisation known to serve a particular population, and so on. At this stage it doesn't filter anything out. It just notices everything that might matter.

Stage 2 — Decide what each mention actually means

Not every mention of a group means the funding request is *for* that group. "In partnership with the Somali Centre" names a partner, not a served population; "expanding beyond our original Indigenous focus" is about the past, not the present; a group listed as one example among many isn't the same as a group the request specifically targets. This stage sorts real evidence of who's served from mentions that only look like evidence.

Stage 3 — Make the call, the same way every time

A single, fixed set of rules turns the sorted evidence into an actual classification. This is the only place a decision gets made – nothing upstream of it can skip ahead and decide early. That's what makes results reproducible: the same input always produces the same output.

> **Why some rows get flagged.** Whenever the evidence is anything less than clear-cut – an ambiguous term, a historical reference, a name-only mention with nothing in the body text to back it up – the tool writes the exact phrase it found into the **Classification Flag** column and leaves the row for a human to check. The philosophy is to flag generously rather than guess silently, especially for anything Indigenous-related, where the tool deliberately over-flags rather than risk a silent miss.

The tool classifies along three axes: **ethnic and cultural origin**, **gender identity**, and **sexual identity**. The specific rules for individual phrasings and edge cases – there are more than a dozen documented case types, from exact keyword matches to country names to BIPOC handling – live in `README.md` section 3 and the working notes in `Documentation/PROCESS.md`. This guide won't repeat them; if you're auditing or refining classification behaviour, read those directly.

2. Three Ways to Run It

All three reach the same numbered menu. Pick whichever fits your situation.

Path	How	Needs
Built executable *Recommended for most staff*	Double-click `ECF Classification.exe` at the top level of the folder	Nothing – no Python, no Docker, no admin rights
Local Python	Double-click `Maintainer\RUN.bat`	System Python installed (builds its own `.venv` on first run)
Docker	`Maintainer\RUN (Docker).bat` or	Docker Desktop installed and running

`Maintainer/run-docker.sh`

Starting it the simple way

Double-click `ECF Classification.exe`. That's the whole setup – nothing to install. A window opens with the numbered menu (Section 6). Type a number, press Enter.

> **Windows may warn you first.** Because this is a new, unsigned program, Windows SmartScreen can show a blue box saying "*Windows protected your PC.*" This is expected for a small in-house tool – it isn't a sign anything is wrong. Click **"More info"**, then **"Run anyway."** You only need to do this once per computer.

Everything else in this guide – where files go, the menu, the working order – applies exactly the same no matter which of the three you use.

3. Where the Files Go

The first time you run the tool, it creates a folder called **ECF Classification** on your Desktop, containing four working folders and a `START HERE.txt`:

```
Desktop/ECF Classification/
    START HERE.txt
    Data Sheets/     README.txt
    Taxonomy/        README.txt
    Gold/            README.txt
    Final Review/    README.txt
```

Every one of those folders has its own README.txt explaining exactly what belongs there and how to name it. Open it in Notepad (just double-click) whenever you're unsure – it's written for exactly that moment.

Folder	What belongs there	Who puts files there
`Data Sheets/`	The funding-request workbooks to classify, one per year	You
`Taxonomy/`	`Taxonomy - Definitions.xlsx` – exactly one file	You
`Gold/`	Hand-audited snapshots of past years	You (optional)
`Final Review/`	Audited copies your corrections get written into	The tool creates this

> **The one naming rule.** Name each workbook with its year:
>
> ```
> Discretionary Funding Requests - 2026.xlsx
> ```
>
> The tool reads the year from the filename and calls that dataset `2026`, automatically pairing it with anything else carrying 2026 – its GOLD file, its copy in Final Review. Next year, drop in `... - 2027.xlsx` and it appears in the menu on its own, with zero setup.

A dataset works fine before its GOLD and Final Review files exist – you don't need to create either by hand. The health check (menu option C) shows "(none yet)" for whatever's still missing. Your READMEs are never overwritten, so any notes you add to them are safe.

> **Already set up?** If the tool's own folder already contains a `Data Sheets` folder with workbooks in it, the tool keeps using those and does **not** create anything new on your Desktop. Existing installations carry on unchanged.

4. What You Provide, What the Tool Creates

Only two files are ever created that you didn't supply. Everything else is written *into* workbooks you provided – which is why the tool asks you to close Excel first.

You provide	Goes in	Needed
`Discretionary Funding Requests - 2026.xlsx`	`Data Sheets/`	Always – this is the workbook being classified
`Taxonomy - Definitions.xlsx`	`Taxonomy/`	Always – nothing runs without it
`FR - 2026 (GOLD-AUDIT).xlsx`	`Gold/`	Optional – only if you already have a hand-audited snapshot of that year

The tool creates	Made by	Where
`Classification Review.xlsx`	Option 2	`Data Sheets/`
`...2026... (GOLD) Final review.xlsx`	Option 3, the first time	`Final Review/`
`stakeholder_dashboard_2026.html`	Option 4	`Data Sheets/`

5. The Six Stages, In Order

1. Put your files in place

Put your workbook in `Data Sheets/`, with the year in its name, and make sure `Taxonomy - Definitions.xlsx` is in `Taxonomy/`.

2. Menu option 1 — Run classification

Fills the ethnic, gender and sexual columns **directly into your workbook**. Nothing new appears – the file you already had gets updated in place. Takes a few minutes.

3. Menu option 2 — Build review workbook

Creates `Data Sheets/Classification Review.xlsx`, containing the rows the engine flagged. Built from that year's GOLD file if you have one; otherwise from the workbook you just classified.

4. Human review

Open `Classification Review.xlsx`, type your corrections into the classification columns, then save and close it.

5. Menu option 3 — Apply my review corrections

Shows you every cell it would change and waits for you to type **YES**. It writes into that year's file in `Final Review/`, creating it for you the first time. Your original workbook is left exactly as the engine wrote it in step 2.

6. Menu option 4 — Build stakeholder dashboard

Reads the `Final Review/` copy, so the dashboard reflects your corrections. Open the resulting `.html` file in a browser.

> Steps 2 to 6 can be repeated as often as you like. Re-running step 2 (option 1) is always safe: it only ever touches the workbook in `Data Sheets/`, never your corrections in `Final Review/`.

6. The Menu

Option	What it does
1. Run classification	Reads the funding-request workbook and fills in the ethnic, gender and sexual-identity columns. Takes a few minutes.
2. Build review workbook	Creates `Data Sheets/Classification Review.xlsx` – the file you type your corrections into. Changes no gold file.
3. Apply my review corrections	Copies your corrections from that review workbook into the Final Review files. Shows you everything it will change first, and asks before writing.
4. Build stakeholder dashboard	Produces the dashboard `.html` file in `Data Sheets`. Open it in a browser.
D. Switch dataset	Cycles through the datasets found in `Data Sheets`. The header always shows which one you're on.
C. Check everything is set up	Confirms the required files are present and nothing is open in Excel. Run this first if something seems wrong.
0. Exit	Closes the tool.

The normal working order

1. **Option 1** – run the classification.
2. **Option 2** – build the review workbook.
3. Open `Data Sheets/Classification Review.xlsx` and type your corrections directly into the classification columns. Save and close it.
4. **Option 3** – apply those corrections. Read the list it shows you, then type `YES` to write them.
5. **Option 4** – build the dashboard, if you need it.

7. Before You Run Anything: Close Excel

> If a workbook is open in Excel, this tool cannot save to it – this applies no matter how you're running the tool. The menu checks for this and will tell you which file to close. Just close it and try again.

8. Letting an AI Assistant Run It For You

Everything above works fine with just the `.exe` and a mouse – this section is for people who'd rather describe what they want in plain language than click through the menu. It's an **alternative**, not a requirement.

What you need first

1. Get this whole folder onto your computer – the same one you'd use to run `ECF Classification.exe`. Nothing in it needs to change.
2. Install **Claude Code**: claude.com/claude-code. It's a separate paid tool from Anthropic, run from a terminal (or from inside VS Code) – it isn't part of this project and isn't required to use the `.exe`. Follow Anthropic's install instructions for your computer.

Open a terminal (or VS Code's terminal panel) **in this folder** and run:

```
claude
```

That drops you into a conversation. Paste one of the prompts below, or describe what you want in your own words.

Run the classification

```
Run the classification on my data. First read HOW TO RUN.md and
Documentation/AI-HANDOFF.md so you understand the tool. Confirm with me
which dataset to use, make sure I've closed the workbook in Excel, then run
the classification step (menu option 1 / Engine_1_and_2/run_all.py) and
show me the summary counts afterwards. Don't change any code.
```

Take me through the whole review cycle

```
Take me through the whole review cycle: run classification, build the
review workbook, tell me where to make my corrections, then apply them and
build the dashboard. Stop and explain each stage before moving to the next.
Don't change any code.
```

Build the dashboard

```
Build or refresh the stakeholder dashboard for my current dataset. Tell me
where the HTML file ends up. Don't change any code.
```

Explain a specific request

```
Explain what the engine did with a specific funding request — [describe or
paste the row]. Walk me through which text triggered which rule, and
whether it was flagged for review. Don't change any code.
```

A safety note. An AI assistant can read and write files just like you can, so the same rules apply: **close Excel first**, and remember that applying your review corrections (option 3) writes into the `Final Review/` copies – read what the assistant proposes to change before you approve it, the same way you'd read option 3's own confirmation list.

9. Troubleshooting

The window tells you what happened in plain language.

Symptom	What it means	What to do
Blue "Windows protected your PC" screen	SmartScreen warning about a new, unsigned program. Expected.	Click "More info", then "Run anyway." Only needed once per computer.
A file is open in Excel	The tool cannot write to a file Excel has locked.	Close it, try again.
"NO DATA FOUND"	No workbook in `Data Sheets`, or it isn't named with a year.	Put one there, named e.g. `Discretionary Funding Requests - 2026.xlsx`.
A file was renamed or moved	The tool expected an exact file it can't find.	Put it back, or pass the message to whoever maintains the tool.
Antivirus quarantined or deleted the `.exe`	Some antivirus tools are suspicious of new, unsigned, single-file programs.	Restore it from quarantine and add an exception, or ask IT to allow it. Re-download/re-copy the file if it was deleted outright.
Docker commands don't respond, or fail immediately	Docker Desktop isn't running – this only applies if you're using the Docker	Start Docker Desktop, wait until it reports it's running, then try the command again.

path.

> Nothing is written unless a step succeeds completely, and option 3 always shows you the full list of changes before it writes anything. If you're unsure, press Enter to cancel – cancelling never changes a file.

If you see a message ending in "Send this message to whoever maintains the tool," take a photo or screenshot of the window. That message contains what's needed to diagnose it.

10. For Whoever Maintains This (Technical)

Everything in this section lives in the `Maintainer/` folder – it holds the build and container tooling, not anything a non-technical user opens.

The .exe — how it's built, and how to rebuild it

Day-to-day use for non-technical staff is `ECF Classification.exe` (at the top level of the folder, beside `Data Sheets/` etc.), built by `Maintainer\build-exe.bat` (PyInstaller, spec file `Maintainer\ECF Classification.spec`) straight from `Maintainer\Tools\launcher.py`. It's a single onefile executable with `Engine_1_and_2/` bundled in as data; PyInstaller extracts it to a temp dir at startup. `Engine_1_and_2/paths.py` has a matching frozen-mode adjustment: the "does an install already have data sitting next to it" check looks at the folder the `.exe` lives in, not the temp extraction dir.

To rebuild after a code change, run `Maintainer\build-exe.bat` (needs `.venv` already set up – run `Maintainer\RUN.bat` once first if not). On success it copies the result to `ECF Classification.exe` at the top level, overwriting any previous copy, ready to hand to staff.

> If a rebuilt exe fails at runtime with `"No module named X"`, add `X` to `hiddenimports` in `Maintainer\ECF Classification.spec` and rebuild – the engine scripts are loaded at runtime, so PyInstaller can't see their imports statically.

The ML layer (`torch`, `sentence_transformers`, `faiss`, etc.) is deliberately excluded from the build – it's ~1 GB of packages the classification path never uses.

RUN.bat

For a maintainer's machine that already has Python. Identical menu, no packaging step; the very first run installs the project's own `.venv`. If it says Python isn't installed, install it from python.org and tick "Add python.exe to PATH" on the installer's first screen.

Docker

For reproducible or server-side runs. `Maintainer\RUN (Docker).bat` / `Maintainer/run-docker.sh` run `docker run -it --rm -v "<workspace>:/data" ecf-discretionary-fr:latest`, which drops into the same menu the `.exe` gives. The image separates code from data: code is baked in at `/app` at build time, and the workspace (`Data Sheets/`, `Taxonomy/`, `Final Review/`) is bind-mounted at `/data` at run time, with `ECF_WORKSPACE=/data` set so `paths.py` resolves straight to the mount. Only `requirements.txt` is installed – the ML extras in `requirements-ml.txt` are excluded on purpose.

Verb-driven, non-interactive runs also work, for scripting or CI (run from the repo root, or adjust the `-f` path):

```
docker build -t ecf-discretionary-fr -f Maintainer/Dockerfile .
docker compose -f Maintainer/docker-compose.yml build
docker compose -f Maintainer/docker-compose.yml run --rm ecf help
docker compose -f Maintainer/docker-compose.yml run --rm ecf classify
docker compose -f Maintainer/docker-compose.yml run --rm ecf preview
docker compose -f Maintainer/docker-compose.yml run --rm -e ECF_DATASET=2023_24 ecf dashboard
```

`docker compose` builds with the repo root as context and mounts `${ECF_WORKSPACE_HOST:-..}` (the repo root, by default) at `/data` – set `ECF_WORKSPACE_HOST` to point it at a different workspace. Excel's file locks apply to the container too: close the workbooks before any command that writes.

Independent entry points

None of the three run paths add classification logic of their own – they all shell out to the same entry points and set `ECF_DATASET`, so the engines remain independently runnable:

```
python Engine_1_and_2/run_all.py
python Engine_1_and_2/Auditing/generate_review_report.py
python Engine_1_and_2/Auditing/apply_review_corrections.py [--apply]
python Engine_1_and_2/Auditing/generate_stakeholder_dashboard.py
```

Full module map, the file-discovery convention, and how to extend a rule are all in `README.md` section 7. Don't re-derive them from the code – they're documented and read faster than they can be reverse-engineered.

> **Regression discipline.** Before an engine change ships, re-run it against the audited gold rows with `Engine_1_and_2/Auditing/regression_audited_rows.py` and confirm no previously-correct row flips. See `regression_baseline.json` and the "Regression discipline" note in `README.md` section 7.

If the .exe ever stops being allowed

Distributing this as an `.exe` through SharePoint depends on a Microsoft 365 tenant policy that can change – most often when IT tightens the rules on unsigned executables. Nothing about the tool depends on that policy: the `.exe` is a convenience for staff, not the product.

Fallback	What it costs
Get it code-signed by IT and carry on as before	The most durable fix – blocking policies almost always target *unsigned* executables, and signing also removes the SmartScreen warning for good. Ask whether IT already holds a signing certificate; many do.
Put the .exe on a network share instead of SharePoint	Nothing changes for staff except where they get the file. Files run from an internal share usually aren't web-marked either, so the SmartScreen warning goes away too.
Zip it and upload that	Often permitted where a bare `.exe` isn't. Staff right-click → "Extract All" before double-clicking.
Skip the .exe entirely – `Maintainer\RUN.bat`	Identical menu, but each machine needs Python installed. Fine for one or two people, poor for a whole team.

Rebuilding the `.exe` at any time is `Maintainer\build-exe.bat` on a machine with the project's `.venv` set up. That's the only step that needs a technical person, and it takes about two minutes.